

# Side-channel attacking browsers through CSS3 features

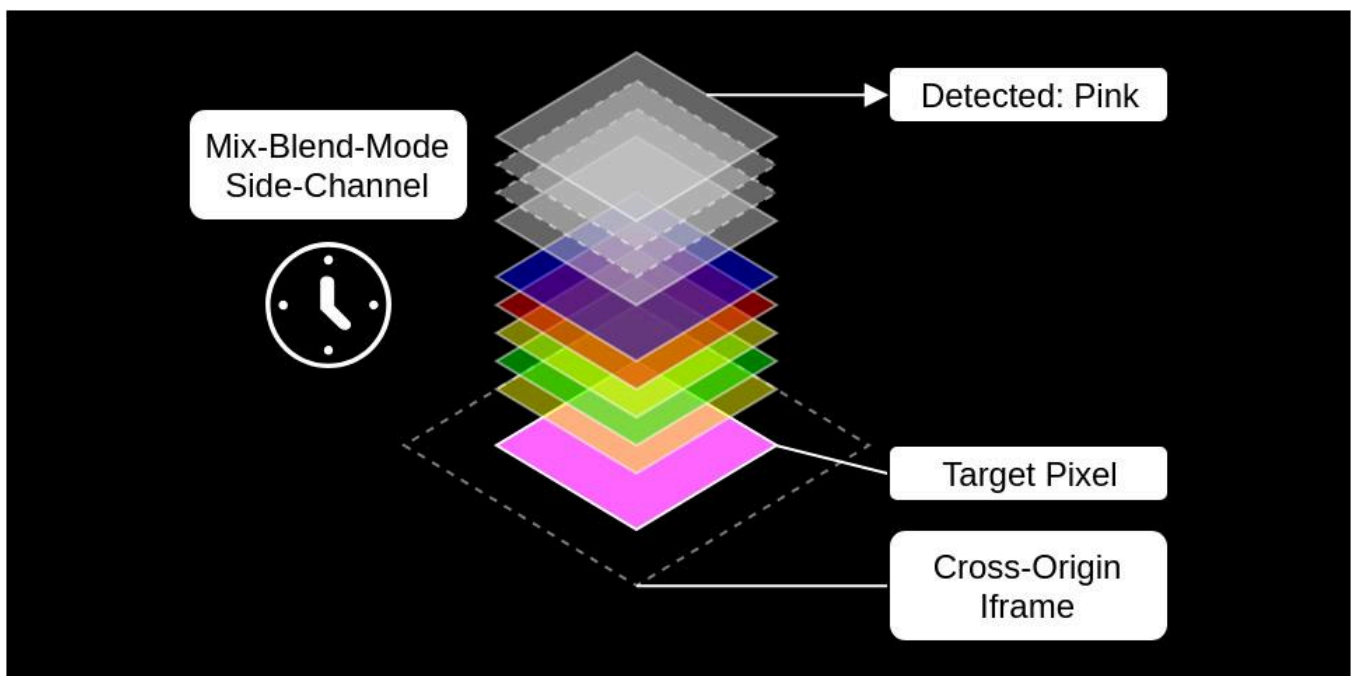
**Side-channel attacking browsers through CSS3 features** - Author not stated, evonide.com.

- Published: date not stated
- Original: <<https://www.evonide.com/side-channel-attacking-browsers-through-css3-features/>>
- Preserved from: <https://www.evonide.com/side-channel-attacking-browsers-through-css3-features/> (live) on 2026-10-02
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

## Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.



(c) [Evonide](#)

With the staggering amount of features that were introduced through HTML5 and CSS3 the attack surface of browsers grew accordingly. Consequently, it is no surprise that interactions between such features can cause unexpected behavior impacting the security

of their users. In this article, we describe such a practical attack and the research behind it.

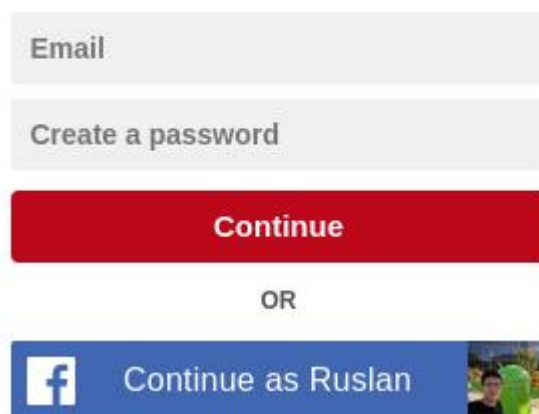
### tl;dr:

- We (co-)discovered a side-channel vulnerability in browser implementations of the CSS3 feature “mix-blend-mode” which allowed to leak visual content from cross-origin iframes.
- We demonstrate the impact of this vulnerability by showing how visiting a malicious site was enough to de-anonymize Facebook users. In particular, exploitation allowed to leak the profile picture, username and likes of unsuspecting visitors all while requiring no additional user interaction.
- This vulnerability affected major browsers like Chrome and Firefox and was disclosed responsibly.

### Credits:

- This work was done in cooperation with Dario Weißer (@haxonaut).
- Max May has discovered the bug independently of us.

## Welcome to Pinterest



### *Pinterest login iframing Facebook*

By chance I stumbled on [Pinterest's homepage](#) which was displaying my Facebook name and picture inside an iframed Facebook button. You would expect that a site like Pinterest can't just read content from the iframe as the same-origin policy would disallow accessing any cross-origin iframe content by default so this should be fine, right?

We wanted to verify if this statement is true given a huge amount of supported features in browsers nowadays. This analysis turned into a bit more research than anticipated and allowed the (co-)discovery of a relatively interesting bug and attack surface which we will elaborate in this article.

### Contents

- 1 Bug Discovery
- 1.1 Attack Setup
- 1.2 Use Cases

- 1.2.1 De-anonymizing Facebook Users
- 1.2.2 Accessing Files behind Access Control Lists
- 2 CSS Blend Modes
- 2.1 Side-Channel Attacking Blend Modes
- 2.1.1 Blend Mode Selection
- 2.2 Leaking Black Pixels
- 2.3 Leaking Precise Colors
- 3 POC
- 4 Misc
- 4.1 Related Work
- 4.2 Responsible Disclosure
- 5 Timeline
- 6 Conclusion

## Bug Discovery

---

Accessing the DOM of an iframe that includes a cross-origin resource is forbidden by default. However, the content of the iframe was displayed in the same context as the rest of the site so we wanted to verify if there is side-channel potential that might allow us to leak state information through the interaction of browser features with the iframed content. With this in mind, Dario and I went ahead and tested various CSS features like “transparency”, “rotation” and “mix-blend-mode” on top of the cross-origin iframe.

By doing so, we discovered a bug that allowed side-channel attacking the CSS feature mix-blend-mode. This feature was introduced beginning 2016 with CSS3 and is available in browsers like Firefox and Chrome. Other browsers like Internet Explorer and Microsoft Edge didn’t support the required feature and Safari didn’t seem to be affected. A full overview of browsers with mix-blend-mode support can be seen in [Mozilla’s Developer Network on mix-blend-mode](#).

After further research, we have discovered that this issue was already reported to the Chromium team and made [temporarily public](#) by accident through a public Chromium auto-cc mailing list in March 7th 2017. We reported this leak and the original thread was made private again. Finally, the bug was made public in the [Chromium bug tracker](#) on 22.02.2018 and was assigned CVE-2017-15417. We have delayed the release of this article as it was just recently fully patched in Firefox 60 so please update your browsers to the newest versions.

## Attack Setup

The discovered side-channel bug allowed to mount the following attack:

- It is possible to overlay the target (cross-origin) iframe with a stack of DIV elements that have the property “mix-blend-mode” enabled.

- The rendering of this stack can then take a variable amount of time depending on the underlying pixel color inside the iframe.
- Finally, by moving this DIV “scan” stack across the iframe, forcing re-renderings and measuring the individual rendering times it is possible to determine the iframe’s content.

Let’s have a look at some real world impact and use cases before considering more bug details.

## Use Cases

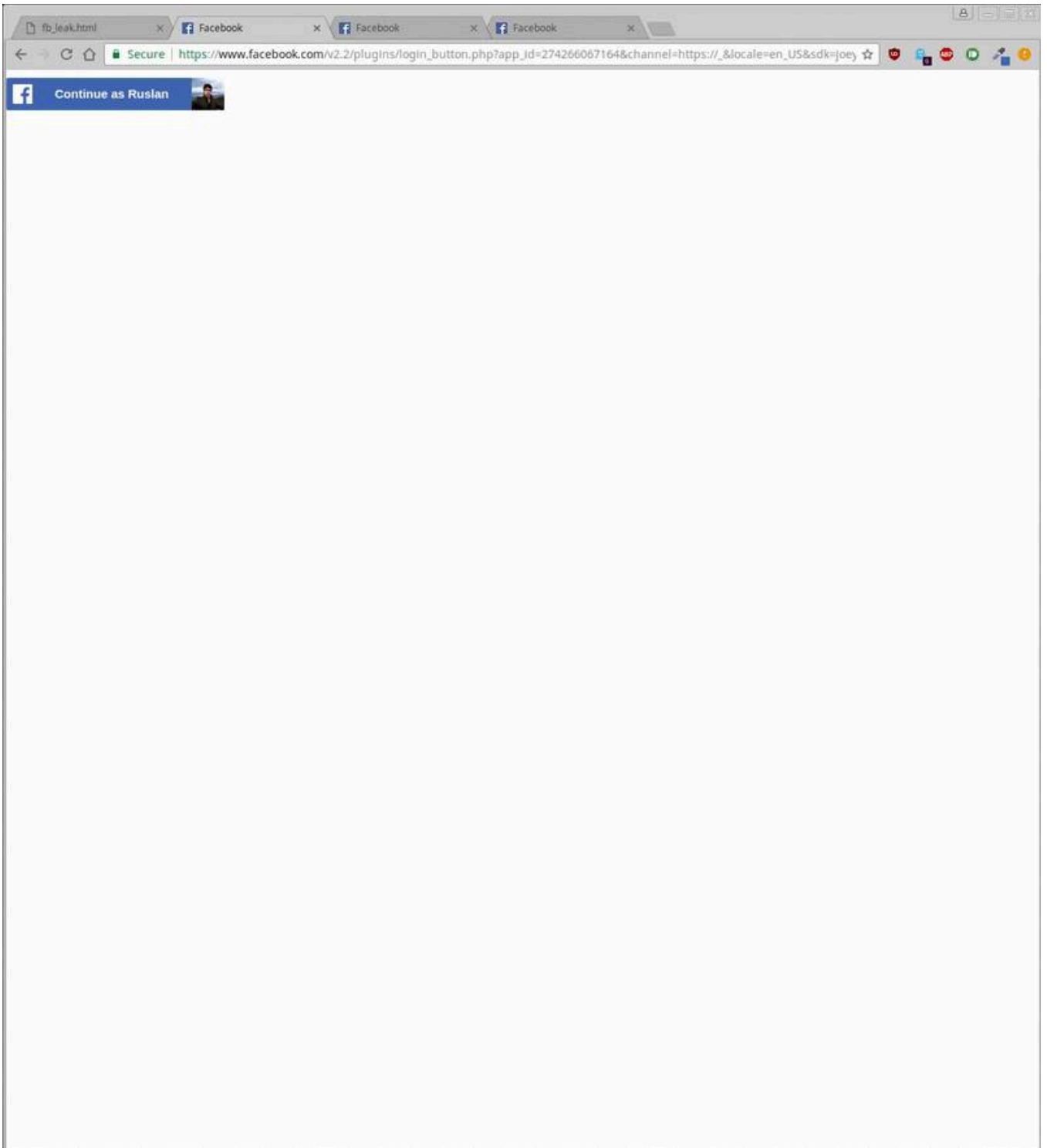
An interesting attack potential lies in obtaining information from websites someone is currently logged into. In particular, we could mount an attack to read information served in iframeable content. Fortunately, most sensitive content like your Facebook message history or your Amazon order history can’t be iframed into other sites that easily.

In order to protect users from attacks like [clickjacking](#), mitigations like Javascript iframe busters and later more solid protections like the HTTP header [X-Frame-Options](#) have been introduced that give users control over which sites are allowed to iframe specific content.

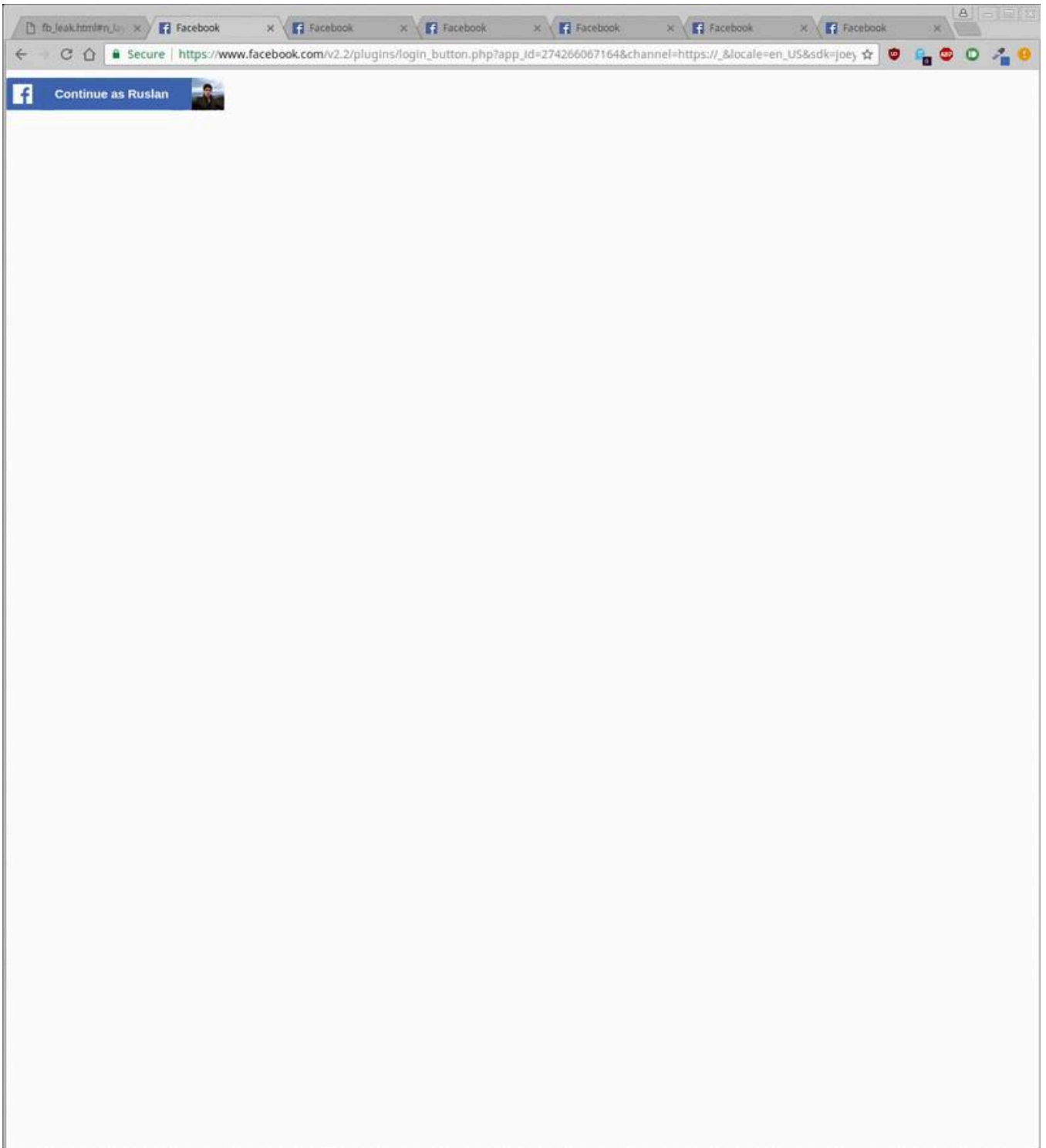
Nevertheless, the Facebook “login” button shows that there still exist iframeable endpoints containing personally identifiable information (PII) you wouldn’t want anyone else to obtain while surfing on other websites.

## De-anonymizing Facebook Users

We constructed a proof of concept HTML file containing a payload for the discovered bug. Opening this file is enough to load different Facebook endpoints inside iframes and to start exploitation which **can be fully camouflaged** as is demonstrated below.



|



||

Leaking the Facebook username (left) and profile picture (right) out of an embedded Facebook Iframe

Overall, exploitation allows to leak the user's name and profile picture as can be seen above. As mentioned, the exploit requires you to previously have logged into Facebook. It then iframes a specific Facebook login endpoint and starts scanning/leaking its content into the local context as can be seen inside the bottom right gray box. Additionally, by

toggleing an animated cat overlay it showcases that this can happen fully in the background without the user taking notice.

It should be noted that Facebook also provides endpoints for:

- Displaying the profile pictures of all friends which have liked the same page as you did with this [page plugin](#).
- Displaying the “like” status of certain pages. A similar POC demonstration for leaking the user’s like status of arbitrary pages can be seen [here](#).

Creating the proof of concept and controlling the side-channel turned out to be quite time consuming so we created a highly experimental and inefficient version. Nevertheless, we were able to conduct the attack with following wall-clock times for Chrome (they were quite similar for Firefox):

- ~20 seconds for leaking the username
- ~5 minutes to leak a vague version of the profile picture
- ~500 milliseconds to check the like status for any given site

Ways to improve performance would have required to implement testing for whole characters instead of leaking individual pixels. Additionally, leaking only a low-resolution black and white version of the profile picture would also improve performance significantly. However, our leaking speed would have been capped at about 1 bit / 16ms (60Hz) due to the JavaScript performance API anyways. As recent results like [Fantastic Timers and Where to Find Them: High-Resolution Microarchitectural Attacks in JavaScript](#) show, there might be ways to improve the exploitation speed for this vulnerability further.

## Accessing Files behind Access Control Lists

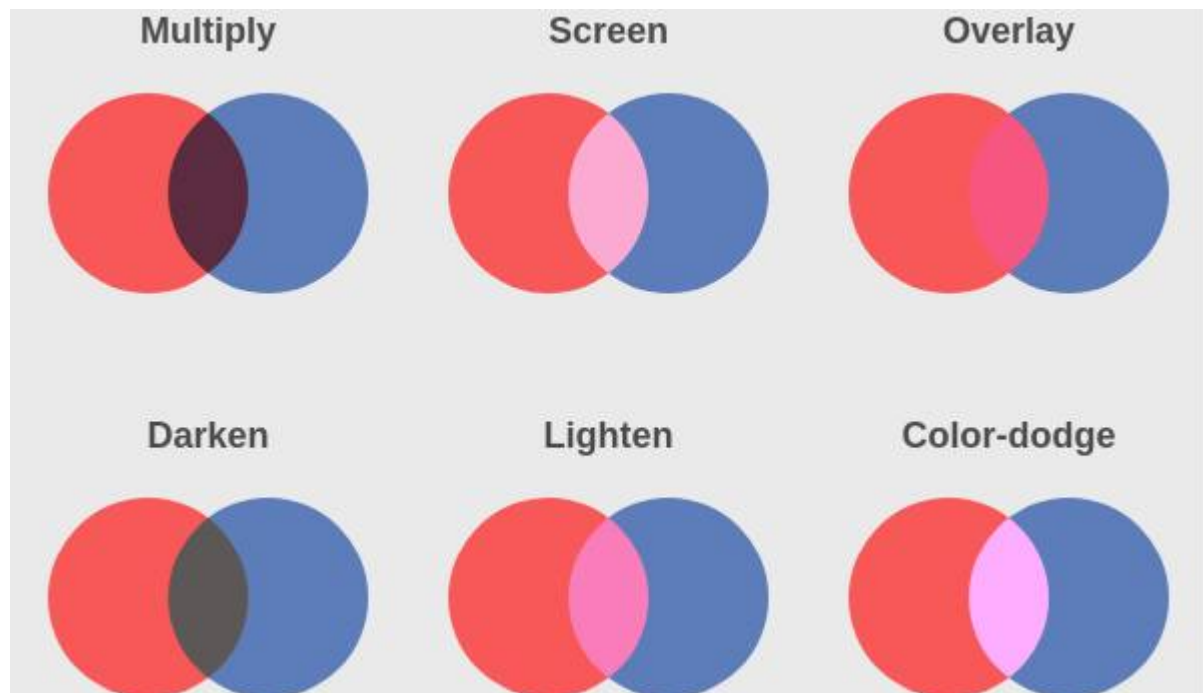
Other use cases include leaking private images, API responses and text files of other sites as these often require to be logged-in but don’t come with an enabled iframe protection.

Overall, use cases like these demonstrate that simple side-channel vulnerabilities can have some relatively unpleasant and wide reaching impact. Let’s take a look at why and how this particular attack works in the first place.

## CSS Blend Modes

---

CSS Blend modes were introduced in 2016 with CSS3, are supported by modern browsers like Chrome/Firefox and allow stacked layers to interact in a multitude of ways. This concept might be known from photo editing programs like Photoshop or Gimp. In particular, there are at least 16 different blend modes whereas the following image visualizes 6 of them:



Visualization of 6 different applied blend modes (see [Blending Modes in CSS: lightenColor Theory and Practical Application](#))

For instance, the blend mode operation “multiply” takes two colors to compute a new color:

- $new\_color = color(bottom\_layer) \times color(top\_layer)$
- A white bottom layer with the color `rgb(255, 255, 255)` and a top layer with the red color `rgb(255, 0, 0)` will result in the color `rgb(255, 0, 0)`. Please also see [Basics of CSS Blend Modes](#) for more details on how the different blend modes operate.

It was quite surprising for us to find out that the blend mode layers were able to interact with cross-origin iframes in the first place so we investigated this further.

## Side-Channel Attacking Blend Modes

The initial question was if color values of an underlying layer like an embedded cross-origin iframe can affect the required rendering time for the blend modes applied in layers above it.

For instance, a simple function like *multiply* is unsuitable as it will always take the same amount of time to compute independent of the underlying content. After some further research and reading blend modes documentation, we found the code that is used to compute more complex blend modes like the non-separable blend mode “saturation”:

```
[...] ClipColor(C) l = Lum(C) n = min(Cred, Cgreen, Cblue) x = max(Cred, Cgreen, Cblue) if
n < 0.0 Cred = l + (((Cred - l) * l) / (l - n)) Cgreen = l + (((Cgreen - l) * l) / (l - n)) Cblue = l
+ (((Cblue - l) * l) / (l - n)) if x > 1.0 Cred = l + (((Cred - l) * (1 - l)) / (x - l)) Cgreen = l +
(((Cgreen - l) * (1 - l)) / (x - l)) Cblue = l + (((Cblue - l) * (1 - l)) / (x - l)) return C SetLum(C, l)
d = l - Lum(C) Cred = Cred + d Cgreen = Cgreen + d Cblue = Cblue + d return
ClipColor(C) [...] SetSat(C, s) if(Cmax > Cmin) Cmid = (((Cmid - Cmin) * s) / (Cmax - Cmin))
```

```
Cmax = s else Cmid = Cmax = 0 Cmin = 0 return C; // Compute the saturation blend mode. Saturation(Cb, Cs) = SetLum(SetSat(Cs, Sat(Cb)), Lum(Cb))
```

|

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

|

[...]

ClipColor(C)

$l = \text{Lum}(C)$

$n = \min(\text{Cred}, \text{Cgreen}, \text{Cblue})$

$x = \max(\text{Cred}, \text{Cgreen}, \text{Cblue})$

if  $n < 0.0$

$\text{Cred} = l + (((\text{Cred} - l) * l) / (l - n))$

$\text{Cgreen} = l + (((\text{Cgreen} - l) * l) / (l - n))$

$\text{Cblue} = l + (((\text{Cblue} - l) * l) / (l - n))$

if  $x > 1.0$

$\text{Cred} = l + (((\text{Cred} - l) * (1 - l)) / (x - l))$

$\text{Cgreen} = l + (((\text{Cgreen} - l) * (1 - l)) / (x - l))$

$\text{Cblue} = l + (((\text{Cblue} - l) * (1 - l)) / (x - l))$

return C

SetLum(C, l)

$d = l - \text{Lum}(C)$

$\text{Cred} = \text{Cred} + d$

$\text{Cgreen} = \text{Cgreen} + d$

$\text{Cblue} = \text{Cblue} + d$

return ClipColor(C)

[...]

SetSat(C, s)

if( $\text{Cmax} > \text{Cmin}$ )

$\text{Cmid} = (((\text{Cmid} - \text{Cmin}) * s) / (\text{Cmax} - \text{Cmin}))$

$\text{Cmax} = s$

else

```

Cmid = Cmax = 0
Cmin = 0
return C;
// Compute the saturation blend mode.
Saturation(Cb, Cs) = SetLum(SetSat(Cs, Sat(Cb)), Lum(Cb))
||

```

While the details of the above code (see [Compositing and Blending W3C Candidate Recommendation](#) for the full code) are irrelevant, you can make two important observations:

- This computation is way more complex than the multiply operation.
- The code uses different branches depending on the given input colors!

The key point is that depending on the given layer colors, the program will execute either more or less instructions which is showing side-channel attack potential. The theory was sound and we were excited but putting it to the test turned out to be a bit more complicated than anticipated.

Particularly, one single computation is not enough to measure any significant rendering time difference but we could stack several thousand layers on top of each other to increase the effect (rendering a stack of superimposed layers can't be parallelized). With the blend mode type set to saturation we finally succeeded: If the stack was placed on a layer with color A, the rendering took twice as long compared to when the layer was of another color B. Bingo: it was clear that pixel colors influence the rendering time and can therefore be leaked in practice, too!

## Blend Mode Selection

In order to better understand the attack surface and to choose a suitable blend mode, we have looked at the graphic library [Skia](#) which is used by browsers like Chrome. Skia is used to draw text, geometries and images and its blend mode source code can be found in the Google Github repository at [skia/blob/master/src/gpu/glsl/GrGLSLBlend.cpp](#).

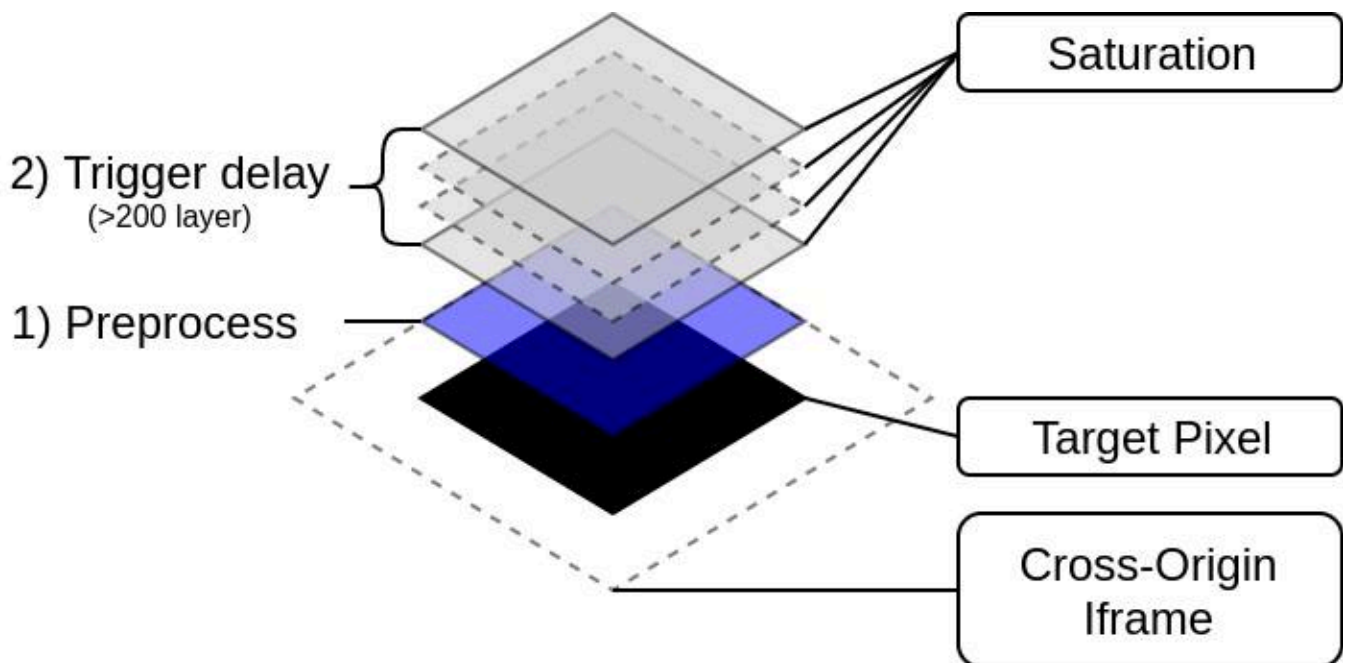
Studying the source code confirmed that the modes "SkBlendMode::kHue" and "SkBlendMode::kSaturation" offered the most computationally intensive code paths by invoking helper functions like `add_sat_function` and `add_lum_function`. After some tests, we confirmed our decision to stick with the "saturation" blend mode.

## Leaking Black Pixels

At this stage, we knew that stacking many layers with an activated saturation blend mode could lead to either a computation heavy chain reaction or to trivial computations depending on the target pixel's color. Particularly, through experimentation we detected that a stack of saturation layers would render slowly if any color channel of the target pixel has a value  $> 10$ .

- **Please note:** In this section we use the words pixel and layer interchangeably as a layer with dimension  $1 \times 1$  counts as a pixel.

While this already sounds interesting we had to add some preprocessing in order to utilize this as a simple binary test which, for example, would allow you to test if a specific target pixel channel is exactly 0 or not:



A tower of “saturation” mix blend mode layers (gray) with an underlying preprocessing stage (blue). In this example, only if the target pixel has a color that matches `rgb(0,_,_)` like `rgb(0,0,0)`, the rendering will take about twice as long compared to when it hasn’t.

The picture above contains a target pixel (black) which lies inside a cross-origin iframe at the bottom, a “preprocessing” stage (blue) and a stack of saturation blend mode layers (gray) on top of it.

- \*\*\*\*Saturation Stack (gray):

This stack consists of many homogeneous layers applying the saturation blend mode operation. The main idea of this stack is to cause a measurable rendering delay if its (underlying) input layer has any color channel with a value  $> 10$ . In any other case, this stack will apply way less computations, render fast and act almost as a no-op in this context.

---

- **Preprocessing Stage *\*(blue):***\*\*\*

\*\*We can’t just directly pass the target pixel inside the iframe to the saturation stack as the pixel can have an arbitrary color and the saturation stack renders slowly only when any of its input pixel color channels has a value  $> 10$ . Hence, all other channels need to be “muted” by multiplying them with 0. This stage computes:

```
preprocess(target_pixel) = rgb(11, 0, 0) // if color(target_pixel) == rgb(0, _, _) rgb(0, 0, 0) //
otherwise
```

|

1

2

|

```
preprocess(target_pixel) = rgb(11, 0, 0) // if color(target_pixel) == rgb(0, _, _)
rgb(0, 0, 0) // otherwise
```

||

With this setup the rendering will be slow only if the target pixel has a color of `rgb(0, _, _)`. However, the value of each color channel must still be retrieved independently from each other. Having such a test is already powerful enough to leak text from an iframe but we wanted to take this one step further to show that you can also leak specific pixel colors.

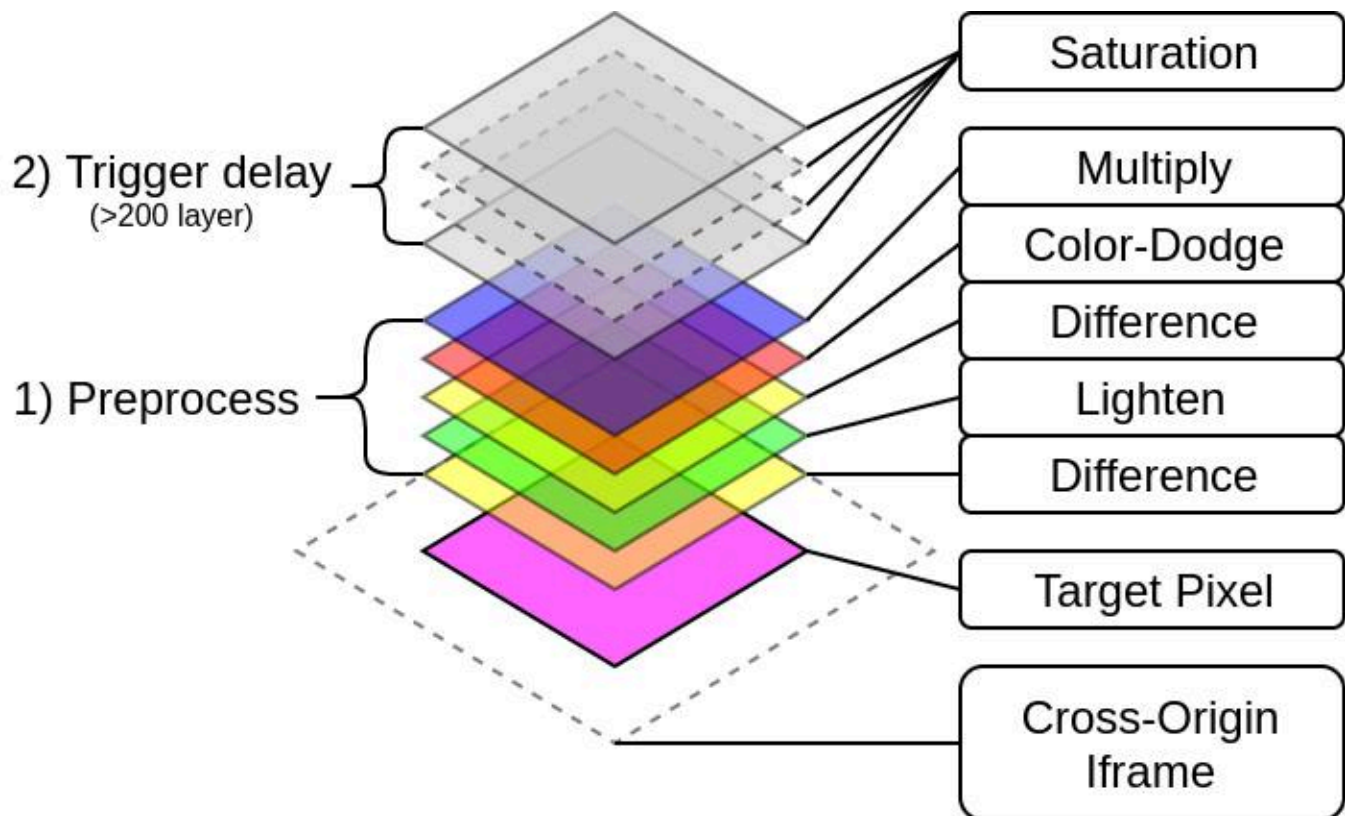
## Leaking Precise Colors

Certainly, you can adjust the setup above to detect if a pixel has a specific color by adjusting the preprocessing but you still wouldn't want to apply  $256 \times 3 = 768$  tests to do so for every single pixel.

Our new goal was to come up with a test that allows to tell if a specific bit in a specific color channel of our target pixel is set. As each color channel has 8 bits and we have 3 channels you would therefore need only 24 tests to leak the exact color. In our tests, leaking the top 4 bits per color channel so overall 12 tests were already enough to get pretty clear results.

Implementing this test required to come up with a multi-layered and more complex preprocessor. Particularly, mixing together different blend modes allows you to come up with a computationally powerful framework. Designing this turned out to be quite a fun exercise: it's quite cool that you can create a program by stacking different blend layers together, isn't it?

Our new resulting "scan tower" can be seen below.



*Mix-blend-mode “scan tower” setup to detect if a specific color channel bit is set in the target pixel.*

As can be seen, this setup is similar to the previous one with the exception that the preprocessing is way more fine grained.

- **\*\*Preprocess Stack:**

We need to apply a multitude of different blend modes to cause a heavy computation chain in the saturation stack only if a specific bit in a specific target pixel color channel is set. **\*\*For example this could compute:**

```
preprocess(target_pixel) = rgb(11, 0, 0) // if 8th bit of red target pixel channel is 1
rgb(0, 0, 0) // otherwise
```

```
|
1
2
|
```

```
preprocess(target_pixel) = rgb(11, 0, 0) // if 8th bit of red target pixel channel is 1
rgb(0, 0, 0) // otherwise
```

```
||
```

Overall, this setup was **quite complex and quirky** so feel free to skip the nitty-gritty details below.

Consider the following example. Let’s say we have a target pixel with the reddish color `rgb(160, 0, 0)`. Our goal is to leak the highest bit of its red channel. The binary

representation would be `rgb(10100000b, 0, 0)` so we would like to detect that the highest bit is a 1. To leak this specific bit we have to adjust the colors of the first three preprocess layers: difference, lighten and difference. In particular, the colors are set as:

- Difference: color set to the sum of all previously leaked bits. We haven't leaked any so this will be `rgb(00000000b, 0, 0)`.
- Lighten: red channel color set to  $2^{(\text{currently tested bit})-1}$ . We want to read the highest bit so this will be  $2^{7-1}=127$  so `rgb(127,0,0)`
- Difference: same color as the lighten layer above so `rgb(127,0,0)`

Preprocess Example: Leaking the highest bit of the target pixel's red channel | Blend-Type | Operation | input\_color | own\_color | Result | | | Difference | `abs(input_color - own_color)` | `rgb(160, 0, 0)` | `rgb(0, 0, 0)` | `rgb(160, 0, 0)` | | | Lighten | `max(own_color, input_color)` | `rgb(160, 0, 0)` | `rgb(127, 0, 0)` | `rgb(160, 0, 0)` | | | Difference | `abs(input_color - own_color)` | `rgb(160, 0, 0)` | `rgb(127, 0, 0)` | `rgb(33, 0, 0)` | | | Color-Dodge | `(input_color == 0) ? 0 : 255` | `rgb(33, 0, 0)` | `rgb(255, 255, 255)` | `rgb(255, 0, 0)` | | | Multiply | `input_color *= rgb(11, 0, 0)` | `rgb(255, 0, 0)` | `rgb(11, 0, 0)` | `rgb(11, 0, 0)` | |

As described before, when our saturation stack is facing the color `rgb(11, 0, 0)` it will render slowly and will disclose that the highest bit is a 1. If we now want to read the second-highest bit we only have to adjust the colors of the first three preprocess layers (difference, lighten and difference) according to the setup above.

Preprocess Example: Leaking the second-highest bit of the target pixel's red channel | Blend-Type | Operation | input\_color | own\_color | Result | | | Difference | `abs(input_color - own_color)` | `rgb(160, 0, 0)` | `rgb(128, 0, 0)` | `rgb(32, 0, 0)` | | | Lighten | `max(own_color, input_color)` | `rgb(32, 0, 0)` | `rgb(63, 0, 0)` | `rgb(63, 0, 0)` | | | Difference | `abs(input_color - own_color)` | `rgb(63, 0, 0)` | `rgb(63, 0, 0)` | `rgb(0, 0, 0)` | | | Color-Dodge | `(input_color == 0) ? 0 : 255` | `rgb(0, 0, 0)` | `rgb(255, 255, 255)` | `rgb(0, 0, 0)` | | | Multiply | `input_color *= rgb(11, 0, 0)` | `rgb(0, 0, 0)` | `rgb(11, 0, 0)` | `rgb(0, 0, 0)` | |

Accordingly, the example above will render fast and imply that the second-highest bit is a 0. By doing so one can read any pixel color up to an arbitrary precision.

## POC

---

- [Facebook Name leak POC](#) (hosted on Github)

This POC helps finding a suitable layer number to improve the side-channel signal.

- [Facebook Name & Picture leak POC](#) (hosted on Github)

This POC demonstrates how a FB picture can be leaked through the side-channel (in addition to the name). However, no guidance for the manual fine-tuning of the layer number is given.

**Please note:** both POCs are highly experimental and overall of relatively poor quality.

## Misc

---

Tests against Safari weren't successful. Upon further investigation we have discovered, that the code to compute the blend modes was already vectorized making the side-channel attack obsolete.

- <https://github.com/WebKit/webkit/blob/master/Source/WebCore/platform/graphics/filters/FilterOperation.cpp>
- <https://github.com/WebKit/webkit/blob/master/Source/WebCore/platform/graphics/ca/cocoa/PlatformCAFiltersCocoa.mm>
- <https://github.com/WebKit/webkit/blob/master/Source/WebCore/css/StyleResolver.cpp>

Similar attacks that have been discovered prior to this work:

- 2013: [Pixel Perfect Timing Attacks with HTML5](#) by Context Information Security

Describes how HTML 5 features like SVG filters and other redraw timing-based attacks could be deployed to leak pixel values from cross-origin sources. The similarity of these issues to our discovered one is quite high.

- 2015: [On Subnormal Floating Point and Abnormal Timing](#) by Andrysco et al.

Showing how timing variability of floating point operations can be utilized to exploit similar attacks via SVG for leaking pixels in the Firefox browser.

- 2016: [lcamtuf's blog – CSS mix-blend-mode is bad for your browsing history

](<https://lcamtuf.blogspot.ch/2016/08/css-mix-blend-mode-is-bad-for-keeping.html>)Describes how mix-blend-modes could be used in a click-jacking like setup to determine the state of visited links. However, the stacking of divs in that context is not related to side-channel attacks.

## Responsible Disclosure

We have contacted Facebook immediately once we had a working proof of concept that was able to de-anonymize their users. However, after multiple messages and clarifying that the discussed vulnerability was not another “clickjacking” attack it became apparent that fixing the vulnerability was infeasible for Facebook as the only fix on their side would have been to remove all endpoints. Simultaneously, we were pushing the Chrome and Skia developers to address this issue in a timely manner. Finally, due to some misunderstandings on our side, reporting the vulnerability to Mozilla was delayed.

The bug was addressed by vectorizing the blend mode computations. Such a vectorized solution can be seen in the Skia code [here](#). Commits fixing the bug and even improving overall rendering speed can be seen [here](#) and [here](#).

## Timeline

---

- 2017-03-07 [First report](#) of the vulnerability by Max May on the Chromium mailing list.
- 2017-05-22 We discovered the vulnerability independently
- 2017-05-25 Vulnerability patched in the Skia rendering library on [Github](#)
- 2017-06-15 Submission of the bug to Facebook's VRP

- 2017-07-27 Facebook's last reply stating that fixing the vulnerability is infeasible as the only option would be to remove the entire feature.
- 2017-11-26 Reported the vulnerability to Mozilla's VRP
- 2017-12-06 Fixed with Chrome version 63.0
- 2018-05-15 Fixed with Firefox Quantum version 60.0

## Conclusion

---

Side-channel vulnerabilities are very sneaky and sometimes hard to patch as was seen with the quite recent and ongoing Meltdown/Spectre discoveries. Similarly, it is no surprise that an ever growing browser feature landscape and more demanding performance for all these features contributes even more to this core problematic in its own way. Particularly, mix-blend-mode is only the tip of the iceberg when it comes to the gigantic rendering feature amount CSS3 and webkit have already introduced.

We have only demonstrated the attack potential against Facebook. However, throughout the web there are tons of other sensitive resources which could be affected by attacks like this in a similar fashion. Unfortunately, we anticipate more and more of such vulnerabilities to be discovered over the years to come.

P.S. You should always protect important resources with the "X-Frame-Options" header set to deny.

Tags [cross-origin](#) [css3](#) [facebook](#) [iframe](#) [mix-blend-mode](#) [side-channel](#)

### About the author

[View All Posts](#)

[image: image]

### Ruslan Habalov

Is dealing with Information Security issues for more than 10 years. Likes sophisticated challenges and is additionally interested in Artificial General Intelligence. [\[Read more...\]](#)

---

Archived from <https://www.evonide.com/side-channel-attacking-browsers-through-css3-features/>. Rendered offline from the local Markdown copy.